

CS595 2026 Project

Team: EmphathAlcs

ScribeGuard

An AI-Powered Ambient Clinical Documentation Platform with OpenMRS Integration

Project Definition Document

Final Submission — End-of-Semester Handoff Package

Project ID	CS595-LOF-2026-001
Team Members	Atishay Jain, Shifaa Nazeersaheb Nadaf, Jayadeep Gopinath, Radhika Sharma, Atharv Patil
LOF Pillar	Patient Engagement (primary)
Last Updated	May 2026
Status	Layer 1 (Scalable MVP) Complete — Documented for new team handoff

1. Project Definition

Project Title – CS595-LOF-2026-001 | ScribeGuard: AI-Powered Ambient Clinical Documentation with OpenMRS Integration

Author(s) – Shifaa Nazeersaheb Nadaf, Jayadeep Gopinath, Radhika Sharma, Atishya Jain, Atharv Patil

LOF Pillar –

- ☒ Patient Engagement
- ☐ Data Analysis & Population Health
- ☐ Medical Education

Last Updated: May 2026 (revised end-of-semester after MVP completion)

2. Introduction

a. Problem Description

Healthcare delivery is burdened by a documentation crisis that directly harms both physicians and patients. Physicians in outpatient and inpatient settings spend an estimated 37–49% of their working time on clinical documentation, with primary care doctors entering data for more than two hours per every hour of direct patient contact. The American Medical Association recognizes this as a systemic issue contributing to physician burnout, which now affects more than half of all practicing clinicians in the United States.

The standard for clinical documentation is the SOAP note (Subjective, Objective, Assessment, Plan), which captures the physician's reasoning about a patient encounter in a structured format that is actionable, billable, and usable by future care providers. Producing SOAP notes manually after every visit is enormously time-consuming. Notes written under time pressure are often rushed, incomplete, or written hours after the encounter when critical details have faded from memory.

In OpenMRS-based health systems — the world's most widely deployed open-source EHR platform, used across hundreds of facilities in the United States, Africa, and Southeast Asia — the problem is compounded because clinical staff must manually transcribe handwritten or dictated notes into structured EHR fields after each encounter. There is no integrated AI mechanism to capture, transcribe, and structure a doctor-patient conversation into a clinical note automatically. Commercial solutions such as Nuance DAX, Suki, and Abridge exist but are expensive, closed-source, and do not integrate with OpenMRS — leaving every OpenMRS-based facility with no ambient documentation option whatsoever.

The Opportunity

ScribeGuard addresses this gap directly. A physician records audio of a patient encounter through a browser-based interface. ScribeGuard transcribes the audio via OpenAI Whisper, passes the transcript to a GPT-4 LLM agent that generates a structured SOAP note, extracts medications and other clinical entities, and presents the result to the physician for explicit review and approval. Only after the physician approves

does ScribeGuard write the finalized note and structured data into the patient's OpenMRS record via FHIR R4. The system is built around a strict physician-in-the-loop principle: no AI output ever reaches the EHR without an explicit human approval action, which is enforced server-side.

b. Purpose

ScribeGuard is an ambient AI clinical documentation assistant that reduces physician documentation burden by automatically transcribing patient encounters and generating structured SOAP notes, while guaranteeing — through a server-enforced approval gate and an immutable audit trail — that no AI output reaches the EHR without explicit physician approval.

Goals & Objectives

- Reduce physician post-encounter documentation time from 2+ hours per day to under 30 minutes by automating SOAP note generation from recorded conversations.
- Eliminate manual re-entry by pushing physician-approved AI-generated notes directly into the patient's OpenMRS record via FHIR R4.
- Provide a physician review-and-edit interface that clearly labels every note as AI-assisted and requires explicit approval before submission, preserving clinical accountability.
- Extract structured clinical entities (medications, allergies, conditions with ICD-10/SNOMED codes, vital signs, follow-ups) and write them as proper FHIR resources rather than free-text.
- Maintain a complete, immutable audit trail of every AI action and physician decision for compliance, quality improvement, and reproducibility.
- Design the architecture so future medication-safety layers (prescription reconciliation, dosage anomaly detection) can be added without re-architecting the core pipeline.
- Deliver a complete, demonstrable, polished MVP within the available sprint window by focusing exclusively on the ambient documentation workflow.

c. Scope

Included in Scope (Layer 1 MVP — Completed)

- Web-based audio recording interface using the browser MediaRecorder API in React (no native mobile app).
- OpenAI Whisper transcription with quality scoring (word count, duration, quality score, detected issues).
- GPT-4 SOAP note generation with version history; previous versions are preserved when notes are regenerated.
- Structured extraction of medications (name, dose, route, frequency, indication), allergies (with severity), conditions (ICD-10/SNOMED), vital signs, and follow-up instructions.
- Physician review workflow: open review → edit any SOAP section → explicit approve/revert state machine, with a per-section edit log.
- OpenMRS FHIR R4 submission creating Encounter, clinical-note Observation, AllergyIntolerance, Condition, MedicationRequest, and vital-sign Observation resources, with returned UUIDs stored back on the corresponding database rows.

- Live patient chart context fetched from OpenMRS at encounter start (demographics, active medications, allergies, conditions, recent encounters).
- Simulation mode (OPENMRS_SIMULATE=true) returning stable fake UUIDs for development/testing without a live OpenMRS server.
- Local AI provider option (faster-whisper + Ollama) requiring no external API keys.
- Full audit trail with append-only audit_events table, agent_runs log, and queryable chronological timeline endpoint.
- React frontend with Dashboard (recording), Encounter Workspace (review/edit/approve/submit), History, and Patient directory pages, plus live agent timeline visualization.
- PostgreSQL schema (13+ tables) managed via 9 Alembic migrations; full Docker Compose configuration.

Excluded from Scope

- Layer 2 — AI Prescription Reconciliation Engine (deferred to future development; architectural extension points are stubbed).
- Layer 3 — Dosage Anomaly Detector (deferred; depends on Layer 2).
- React Native mobile application (descoped at the proposal-revision stage in favor of a web-only approach).
- E-prescribing, automated prescription writing, or direct pharmacy transmission.
- Real-time / streaming live transcription (audio is recorded then processed post-encounter).
- Full HIPAA-compliant production deployment with encryption at rest, formal compliance audit, and audit-log shipping to immutable storage.
- Multi-tenant or multi-clinic deployment, authentication beyond a stub login page, and production secrets management.
- Insurance billing, claims processing, and prior authorization workflows.
- Specialty-specific clinical pathways (oncology dosing, pediatric weight-band dosing, psychiatry workflows).
- Custom LLM fine-tuning or training from scratch (pre-trained GPT-4 family models with prompt engineering are used).

d. Audience

ScribeGuard targets four user groups across the clinical workflow and administrative chain. Each is defined below with the characteristics most relevant to product decisions.

1. Physicians & Residents (Primary Users)

- Attending physicians, hospitalists, and medical residents seeing 15–30 patients per day in outpatient or inpatient clinic settings.
- Clinical documentation is their single largest administrative burden — 2+ hours per day on note-writing.
- Technically comfortable with web applications, but highly resistant to workflow disruption. The interface must integrate naturally into the post-encounter routine without adding friction.

- Require full trust and control over AI outputs: every note is clearly labeled as AI-assisted, fully editable before submission, and never auto-committed.
- Speed is critical: the full audio-to-submission workflow must complete in under three minutes for a typical 10-minute recording.

2. Clinical Pharmacists & Care Coordinators (Secondary Users)

- Responsible for medication safety review, polypharmacy management, and care plan coordination.
- In the current MVP they benefit indirectly from higher-quality, structured medication lists in OpenMRS.
- In future Layers 2–3 they will become primary users of the dashboard's reconciliation findings and dosage alerts; the data pipeline already produces structured medication output ready for that handoff.

3. Medical Students & Residents (Tertiary Users — Education Context)

- Junior clinicians who can use AI-generated SOAP notes as structured examples of correct clinical documentation format and reasoning.
- The physician edit-and-review workflow models best practices for reviewing AI output in a clinical setting — a critical skill for the next generation of clinicians.

4. Compliance Officers / OpenMRS System Administrators (Technical Users)

- IT and compliance staff responsible for maintaining the OpenMRS instance, managing API credentials, and auditing the documentation process.
- Require clean OpenAPI/Swagger documentation for all ScribeGuard REST endpoints, structured error logging, and an immutable audit trail of every AI action and physician decision.
- Must be able to inspect which model version and prompt version generated each note for reproducibility and post-hoc review.

e. Overview

ScribeGuard is delivered as a React 19 + TypeScript frontend, a Python 3.11 / FastAPI backend, and a PostgreSQL 16 database, integrated with an OpenMRS EHR instance via the FHIR2 R4 module. The backend implements a multi-agent AI architecture: each step in the clinical documentation pipeline is handled by a specialized autonomous agent, coordinated by a central orchestrator that handles retries, advances pipeline state, and writes audit events.

End-to-End Workflow

Step 1 — Audio Capture. The physician opens the ScribeGuard web application after a patient encounter, selects the patient, and uses the browser's native MediaRecorder API to record (or upload) the encounter audio. No mobile app installation is required. The audio file is transmitted to the FastAPI backend and saved to disk under the configured AUDIO_STORAGE_DIR.

Step 2 — Whisper Transcription. The TranscriptionAgent calls OpenAI's Whisper API, producing a full text transcript and quality metadata (word count, duration, quality score, detected issues such as silence or low confidence regions). The transcript is stored in PostgreSQL alongside audio metadata.

Step 3 — GPT-4 SOAP Note Generation. The ClinicalNoteGenerationAgent passes the transcript to a GPT-4 family model with a version-pinned system prompt that instructs the model to produce a structured SOAP note: Subjective (patient-reported symptoms and history), Objective (examination findings, vitals, lab values), Assessment (diagnoses and differential), and Plan (treatments, prescriptions, referrals, follow-up). The note is persisted as a versioned record; previous versions are preserved when the note is regenerated.

Step 4 — Structured Entity Extraction. The MedicationExtractionAgent and supporting extractors parse the SOAP note to produce structured medication, allergy, condition (with ICD-10 / SNOMED codes), vital sign, and follow-up records. These are stored in dedicated tables, ready for both the physician review UI and downstream FHIR mapping.

Step 5 — Physician Review & Edit. The generated SOAP note is displayed in the Encounter Workspace with a clear “AI-Generated — Pending Review” label. The physician can edit any section inline; every edit captures the original and edited text in the physician_edits table. The interface also displays the extracted medication list, allergies, conditions, and vital signs. The physician must explicitly click “Approve & Lock” to advance — no note is ever automatically committed. Approval can be reverted any time before submission.

Step 6 — OpenMRS Write-Back via FHIR R4. Upon physician approval and an explicit Submit action, the OpenMRSIntegrationAgent (a composite of auth → patient context → encounter mapper → resource writer → verifier sub-agents) authenticates against OpenMRS, resolves the patient, maps the approved data to FHIR R4 resources (Encounter, clinical-note Observation, AllergyIntolerance, Condition, MedicationRequest, vital-sign Observations), POSTs them, and verifies the writes by reading back the created resources. Returned UUIDs are stored on the corresponding database rows.

Step 7 — Audit Aggregation. The AuditTraceabilityAgent aggregates all event-stream entries into a queryable chronological timeline. Every agent invocation, physician edit, approval, revert, and submission is logged in the immutable audit_events table. The complete history — audio, transcript, every SOAP version, every physician edit, the approval, the submission, and all agent run logs — is retained in PostgreSQL and exposed via the audit timeline API.

Future Architecture (Roadmap)

The structured medication output produced in Step 4 is the designed entry point for Layer 2 (Prescription Reconciliation Agent) and Layer 3 (Dosage Anomaly Detector). The PostgreSQL schema, the agent registry, and the OpenMRS integration layer were designed from the start to support those extensions without architectural changes — a new agent simply needs to be registered with the orchestrator and given a result table.

f. Glossary / Terminology

Term	Definition
SOAP Note	Structured clinical note format: Subjective, Objective, Assessment, Plan.
FHIR R4	Fast Healthcare Interoperability Resources, Release 4 — the standard API/data format for EHR integration.
OpenMRS	Open Medical Record System — the open-source EHR platform ScribeGuard integrates with.
FHIR2 Module	OpenMRS module that exposes patient data as FHIR R4 resources; required for ScribeGuard integration.
Whisper	OpenAI's speech-to-text model used for transcription.
Agent	An autonomous software component with a single, well-defined responsibility in the pipeline.
Orchestrator	The central component that coordinates agents, handles retries, and advances pipeline state.
AgentContext	Shared state bundle (DB session, encounter, repositories) passed into every agent run.
Processing Stage	Fine-grained pipeline state (e.g. transcribing, soap_drafted, in_review, submitted).
Encounter Status	User-visible lifecycle state: pending → approved → pushed → failed.
SOAP Version	Each regeneration creates a new version; is_current=true marks the active draft.
Simulation Mode	OPENMRS_SIMULATE=true — agent returns fake UUIDs without real FHIR calls.
CIEL	Clinical Interface Engine for Languages — medical concept dictionary used in OpenMRS.
ICD-10	International Classification of Diseases, 10th rev. — used for diagnosis coding.
SNOMED CT	Systematized Nomenclature of Medicine — used alongside ICD-10 for condition coding.
RxNorm	U.S. National Library of Medicine medication terminology (planned for Layer 2).
Beers Criteria	Reference list of potentially inappropriate medications in older adults (planned for Layer 3).
eGFR	Estimated Glomerular Filtration Rate — kidney function measure used in dose adjustment (Layer 3).
Alembic	SQLAlchemy's database migration tool; 9 migrations make up the schema.
Pydantic v2	Python data validation library used for API schemas and configuration.
MediaRecorder API	Browser-native audio capture API used by the React frontend.

g. Skills Needed

Backend Development

- Python 3.11+ (async/await, typing, Pydantic v2).
- FastAPI framework and dependency injection patterns.
- SQLAlchemy 2.0 ORM and the repository pattern used throughout this codebase.
- Alembic migrations (the 9 existing migrations must be applied in order via alembic upgrade head).
- PostgreSQL — schema design, indexes, basic query tuning.

AI / ML

- OpenAI API usage for Whisper transcription and GPT-4 family chat completions.
- Prompt engineering with structured JSON output in a clinical domain.
- Optional: faster-whisper and Ollama for fully local, free, offline inference.

Healthcare / EHR Integration

- FHIR R4 resource types: Encounter, Observation, Condition, AllergyIntolerance, MedicationRequest.
- OpenMRS data model and the FHIR2 module.
- Basic clinical terminology: SOAP, ICD-10, SNOMED CT, CIEL, RxNorm.

Frontend Development

- React 19 with TypeScript strict mode.
- React Router v7 for client-side routing.
- Browser MediaRecorder API for audio capture.
- Vite as the build tool.

DevOps

- Docker / Docker Compose for local development.
- Basic Linux/shell skills for environment setup.

LOF Partner Involvement

OpenMRS community for FHIR2 module questions and reference dataset access. Any clinical partner providing physician access for usability testing of the review workflow.

3. System Overview

a. Architecture

ScribeGuard uses a multi-agent pipeline architecture organized into four horizontal layers. The frontend talks only to the FastAPI HTTP API; the API never invokes agents directly but delegates everything to the central orchestrator, which in turn coordinates seven specialized agents. The orchestrator is the only code in the system that advances `encounter.processing_stage` across stage boundaries — agents may set transient flags but never own pipeline state.

Figure 3.1 — High-Level Architecture

React Web Frontend — Dashboard Encounter Workspace History Patients
↓ HTTP (fetch / REST, JSON)
FastAPI HTTP API — 11 routers (encounters, pipeline, review, submission, audit, agents, ...)
↓
Agent Orchestrator — retries, state transitions, audit emission, registry
↓ coordinates 7 specialized agents in sequence
Intake → Transcription → ClinicalNoteGen → MedicationExtraction → PhysicianReview → OpenMRSIntegration → AuditTraceability
↓ ↓
PostgreSQL (13+ tables: encounters, transcripts, soap_notes, medications, audit_events, agent_runs ...) External: OpenAI Whisper + GPT-4, OpenMRS FHIR R4 (or Ollama + faster-whisper for local)

Key Architectural Decisions

- **Dedicated agents instead of a single pipeline function** — each agent has its own failure mode (Whisper rate limits, GPT JSON parse errors, OpenMRS auth errors). Typed agents make those boundaries addressable individually with clean retry semantics.
- **Repository pattern** — agents never touch SQLAlchemy sessions directly. They use `ctx.transcripts`, `ctx.soap_notes`, etc. (lazy repository accessors on `AgentContext`), which keeps agents testable in isolation.
- **Explicit orchestrator as single state source** — without a central orchestrator, audit gaps appear when agents partially succeed. The orchestrator is the only code that advances `encounter.processing_stage` across stage boundaries.
- **Versioned SOAP notes** — regenerating a draft after physician edits must not destroy their work. Previous versions are marked superseded; the new one becomes `is_current=true`.
- **OpenMRS split into sub-agents** — auth, patient context, encounter mapping, resource writing, and verification are independent sub-agents so the failure reason is distinct in the audit trail.
- **Server-side enforcement of the approval gate** — OpenMRS submission is rejected at the API layer unless `encounter.status == 'approved'`. The frontend cannot bypass it.

b. Technologies Used

Layer	Technology	Version
Frontend framework	React	19.x
Frontend language	TypeScript (strict mode)	5.x

Frontend build tool	Vite	5.x
Frontend routing	React Router DOM	7.x
Frontend icons	Lucide React	Latest
Backend framework	FastAPI	0.115
Backend runtime	Python	3.11+
Backend server	Uvicorn	0.30
ORM	SQLAlchemy	2.0
Migrations	Alembic	1.13
Data validation	Pydantic v2	2.9
Database	PostgreSQL	16
Database driver	psycopg2-binary	2.9
AI: transcription	OpenAI Whisper (whisper-1)	via openai SDK ≥ 1.54
AI: SOAP generation	OpenAI GPT-4o-mini (configurable)	via openai SDK
AI: local alternative	faster-whisper + Ollama	$\geq 1.0.3$ / latest
FHIR HTTP client	httpx	≥ 0.27
Retry logic	tenacity	≥ 9.0
Structured logging	structlog	≥ 24.4
PDF/DOCX parsing	pypdf, python-docx	≥ 4.0 / ≥ 1.1
PDF generation	fpdf2	≥ 2.8
Containerization	Docker / Docker Compose	Latest

c. Dependencies

External Services

- **OpenAI API** — Whisper transcription + GPT-4 SOAP and medication generation. Requires a paid API key. A typical development session of 10–20 test encounters costs under \$1.
- **OpenMRS FHIR2 API** — the EHR integration endpoint. OpenMRS must have the FHIR2 module installed and enabled. Default endpoint <http://localhost:8080/openmrs/ws/fhir2/R4>. Optional in development via `OPENMRS_SIMULATE=true`.
- **Ollama (optional)** — local LLM server for offline / free inference. Requires separate installation and a pulled model (default: llama3.2).

Internal Dependencies

- PostgreSQL must be running before the backend can start (docker compose up -d db).
- Alembic migrations must be applied before the first API request (alembic upgrade head).
- Frontend assumes the backend is reachable at <http://localhost:8000> by default.
- `CORS_ORIGINS` in `backend/.env` must include the frontend origin (default <http://localhost:5173>).

Future-Layer Dependencies (Documented for Roadmap)

- RxNorm — for medication name normalization in Layer 2 (free public API).
- OpenFDA Drug Labeling API — for dosage thresholds in Layer 3 (free public API; intentionally chosen over commercial alternatives).
- DrugBank — optional richer drug-interaction data in Layer 3 (requires a license).

d. Hardware and Software Requirements

Development Machine

Resource	Requirement
Operating System	macOS, Linux, or Windows (WSL2 recommended for Windows)
RAM	8 GB minimum, 16 GB recommended (especially if using local Whisper)
Disk	5 GB free, more if storing many long audio files
CPU	Any modern x86-64 processor
GPU	Not required for SERVICE_PROVIDER=openai; helpful for SERVICE_PROVIDER=local
Browser	Chrome, Firefox, or Safari with microphone access for audio capture

Required Software

- Python 3.11+
- Node.js 18+
- Docker Desktop (for the PostgreSQL container)
- Git

Recommended

- VS Code or PyCharm for development
- Postman or the built-in Swagger UI (<http://localhost:8000/docs>) for API testing

Production Deployment (Not Yet Configured — see Open Questions)

- Linux VPS or container orchestration (Kubernetes, ECS) with persistent volumes for PostgreSQL data and audio storage.
- Reverse proxy (Nginx, Caddy) with TLS termination.
- CORS_ORIGINS updated to the production domain.
- Real authentication (OAuth2 / OpenID Connect with OpenMRS or similar) replacing the current stub login.

e. Data Description and Sources

Data Generated by the System

- **Audio files** — browser MediaRecorder output (WebM / OGG / MP4 depending on browser). Stored on the backend filesystem at AUDIO_STORAGE_DIR; not stored in the database.
- **Transcripts** — plain-text Whisper output with quality metadata (word count, duration, quality score, detected issues). Stored in the transcripts table.
- **SOAP Notes** — structured JSON with four text sections (Subjective, Objective, Assessment, Plan), versioned. All versions retained. Stored in soap_notes.
- **Clinical Entities** — medications, allergies, conditions (with ICD-10 / SNOMED), vital signs, follow-ups; one table per entity type.
- **Audit Events** — immutable, append-only log of every meaningful system action. Stored in audit_events; never deleted.
- **Agent Run Logs** — timing, status, attempt count, input/output summaries, and error context for every agent invocation. Stored in agent_runs.
- **Submission Records** — OpenMRS write-attempt history including returned FHIR resource UUIDs and any failure reasons. Stored in submission_records.

Data Sources

- Encounter audio: live capture from physician's browser microphone.
- Patient demographics, recent medications, allergies, conditions, encounters: fetched live from OpenMRS via FHIR R4 at encounter start, cached in patient_contexts.
- Clinical entity values: extracted by AI from the transcript and SOAP note.
- Test data: OpenMRS Reference Application demo dataset (synthetic patient records) plus the test_transcripts/ directory of sample audio.

Data Accessibility

- All data is stored in a single PostgreSQL database; accessible to any team member with the DATABASE_URL.
- Audio files are stored on the backend host filesystem under AUDIO_STORAGE_DIR; accessible via the file system or the export endpoints.
- No external data sources require authentication beyond OpenAI (API key) and OpenMRS (username/password).
- Sample/test data: test_transcripts/ for pipeline testing; seed.py to populate the database with test encounters; OPENMRS_SIMULATE=true for end-to-end testing without a live OpenMRS instance.

f. Pre-Work / Setup

Before the project can be built and run, the following must be in place. The complete step-by-step setup is in NEW_TEAM_HANDOFF.md Section 4; the items below are the prerequisites that must be true before that guide will succeed.

- OpenAI API account and key created at platform.openai.com (pay-as-you-go; ~\$1 covers a development session of 10–20 test encounters).
- Docker Desktop installed and running (used for the PostgreSQL container and optionally for the backend).
- Python 3.11+ and Node.js 18+ installed.
- backend/.env file created with the values templated in NEW_TEAM_HANDOFF.md Section 4 (DATABASE_URL, OPENAI_API_KEY, FHIR_SERVER, OPENMRS_SIMULATE, CORS_ORIGINS, etc.).
- Alembic migrations applied: alembic upgrade head from inside backend/.
- OpenMRS instance (optional): for development OPENMRS_SIMULATE=true bypasses this. For real integration, run the OpenMRS Reference Application Docker image and wait ~5 minutes for full startup.
- OpenMRS UUIDs configured if not using the Reference Application defaults (OPENMRS_DEFAULT_PRACTITIONER_UUID, OPENMRS_DEFAULT_LOCATION_UUID).

4. Functional Requirements

a. List of User Tasks

Tasks are expressed as Verb + Noun for usability-testing alignment, per the template guidance.

#	Task	Actor
1	Record Encounter	Physician
2	Upload Audio File	Physician
3	Review Transcript	Physician
4	Edit SOAP Note	Physician
5	Approve SOAP Note	Physician
6	Revert Approval	Physician
7	Submit to OpenMRS	Physician
8	View Audit Timeline	Physician / QA
9	View Encounter History	Physician / Admin
10	Search Encounters	Physician / Admin
11	View Patient Context	Physician
12	View Agent Run Status	Physician / Admin
13	Add Medication Manually	Physician
14	Edit Medication	Physician
15	Delete Medication	Physician
16	Export Encounter Data	Admin

b. User Stories

As a physician, I want to:

- Record a patient encounter using my laptop or tablet microphone so that I do not have to type notes during the visit.
- See a SOAP note generated from the recording within 2–3 minutes so I can quickly verify and approve it.
- Edit any section of the AI-generated SOAP note before approval so that I can correct inaccuracies.
- See low-confidence sections highlighted so I know where to focus my review.
- Explicitly approve the note before it is submitted to the EHR so I maintain medical-legal control over the record.
- View the patient's current medications, allergies, and conditions from OpenMRS so I have chart context during review.
- See a live status indicator showing where in the processing pipeline my recording is.
- Review the complete history of all my encounters so I can find past notes quickly.

As a compliance officer, I want to:

- See a complete, timestamped log of every AI action and physician decision so I can audit the documentation process.
- Know which version of the AI model and prompt generated each note so I can assess reproducibility.

- See a record of all physician edits (original text vs. edited text, by section) so I can assess how much the AI output was modified.

As an OpenMRS administrator, I want to:

- Inspect the FHIR resource UUIDs ScribeGuard wrote so I can correlate them in the OpenMRS database.
- See a clear failure reason in the submission history when a write fails, so I can debug auth or mapping issues.

c. Use Cases

UC-01 — Record and Process Encounter

- Actor: Physician.
- Precondition: Physician has navigated to Dashboard and selected a patient.
- Flow: Click “Start Recording” → conduct encounter → click “Stop” → system uploads audio and auto-runs the full pipeline (Intake → Transcription → SOAP → Medication Extraction).
- Postcondition: Encounter is in `ready_for_review` stage; SOAP note and clinical entities are available for review.

UC-02 — Review and Approve SOAP Note

- Actor: Physician.
- Precondition: Encounter is in `ready_for_review` stage.
- Flow: Navigate to Encounter Workspace → click “Open Review” → review SOAP sections, edit if needed → click “Approve & Lock”.
- Postcondition: `encounter.status == approved`; per-section edit log persisted; SOAP note status is approved.

UC-03 — Submit to OpenMRS

- Actor: Physician.
- Precondition: `encounter.status == approved` (server-enforced).
- Flow: Click “Submit to OpenMRS” → system runs `OpenMRSIntegrationAgent` (auth → patient lookup → FHIR mapping → write → verify) → submission record created with returned UUIDs.
- Postcondition: `encounter.status == pushed`; FHIR resources exist in OpenMRS with UUIDs persisted in the database.

UC-04 — Revert and Re-Approve

- Actor: Physician.
- Precondition: `encounter.status == approved` but not yet submitted.
- Flow: Click “Revert Approval” → edit additional sections → re-approve.
- Postcondition: New approval record created; previous approval marked superseded.

UC-05 — View Audit Timeline

- Actor: Physician / Compliance Officer.

- Precondition: An encounter exists.
- Flow: Navigate to Encounter Workspace → select “Audit” tab → view chronological timeline of all events.
- Postcondition: Full clinical event history displayed with timestamps, actors, and details.

d. Prioritization

Priority	Requirement	Status
P0 — Must Have	Audio capture and upload	Complete
P0 — Must Have	Whisper transcription	Complete
P0 — Must Have	GPT-4 SOAP note generation	Complete
P0 — Must Have	Physician review and approval gate	Complete
P0 — Must Have	OpenMRS FHIR R4 submission	Complete
P0 — Must Have	Immutable audit trail	Complete
P1 — Should Have	Medication extraction	Complete
P1 — Should Have	Allergy, condition, vital sign extraction	Complete
P1 — Should Have	Patient chart context from OpenMRS	Complete
P1 — Should Have	Agent run timeline visualization	Complete
P2 — Could Have	Prescription reconciliation (Layer 2)	Not started
P2 — Could Have	Dosage anomaly detection (Layer 3)	Not started
P3 — Future	Mobile app (React Native)	Descoped
P3 — Future	E-prescribing integration	Not started
P3 — Future	HIPAA audit log export	Not started

e. List of Functional Requirements

ID	Requirement	Priority
FR-01	The system shall capture audio from the physician's browser microphone and upload it to the backend.	P0
FR-02	The system shall transcribe uploaded audio using Whisper and store the transcript with quality metadata.	P0
FR-03	The system shall generate a structured SOAP note from the transcript using a GPT-4 family model.	P0
FR-04	The system shall preserve all versions of SOAP notes; previous versions shall not be deleted when a new version is generated.	P0
FR-05	The system shall extract structured medication entities (name, dose, route, frequency, indication) from the SOAP Plan section.	P1
FR-06	The system shall extract allergies, conditions (with ICD-10/SNOMED), vital signs, and follow-up instructions from the SOAP note.	P1
FR-07	The system shall allow a physician to open a review phase, edit any SOAP section, and record both the original and edited text.	P0
FR-08	The system shall require an explicit physician approval action before any EHR submission is permitted. This requirement shall be enforced server-side.	P0
FR-09	The system shall allow a physician to revert an approval if no submission has yet occurred.	P1
FR-10	The system shall submit the approved encounter to OpenMRS via FHIR R4, creating Encounter, Observation (note + vitals), AllergyIntolerance, Condition, and MedicationRequest resources.	P0
FR-11	The system shall store the FHIR resource UUIDs returned by OpenMRS on the corresponding database rows.	P0

FR-12	The system shall log every agent invocation (status, timing, attempt count, error context) and every clinical/business event in an immutable audit trail.	P0
FR-13	The system shall expose the audit trail as a queryable, chronological timeline via the HTTP API.	P1
FR-14	The system shall fetch and display the patient's current chart context (demographics, active medications, allergies, conditions, recent encounters) from OpenMRS at encounter start.	P1
FR-15	The system shall surface the current pipeline stage to the physician in real time via the frontend.	P1
FR-16	The system shall support a simulation mode (OPENMRS_SIMULATE=true) that skips real OpenMRS calls and returns stable fake UUIDs for development/CI purposes.	P1
FR-17	The system shall support a local AI provider option (faster-whisper + Ollama) that requires no external API keys.	P2
FR-18	The system shall allow physicians to manually add, edit, and delete medication entries from the encounter record.	P1
FR-19	The system shall expose all encounter data (encounters, transcripts, SOAP notes, medications, audit events) via a documented REST API with Swagger UI.	P0
FR-20	The system shall retain a submission history showing every attempt to submit to OpenMRS, including failures and the reason for failure.	P1

f. Data Structures and Interfaces

Database Schema (PostgreSQL — 13 tables)

Table	Purpose
encounters	Root record for each patient visit; tracks processing_stage and user-visible status.
transcripts	Whisper transcription output and quality signals.
soap_notes	Versioned SOAP drafts. Exactly one row per encounter has is_current=true at any time.
medications	Structured drug entities extracted from the Plan section (and manually added).
allergies	Allergy records with severity.
conditions	Diagnosis records with ICD-10 and SNOMED codes.
vital_signs	Vital observations parsed from the SOAP note.
follow_ups	Follow-up instructions from the Plan section.
physician_edits	Per-section edit log (original text vs. edited text).
physician_approvals	Approval records (one current; previous marked superseded on revert).
submission_records	OpenMRS write attempts with returned FHIR UUIDs and failure reasons.
agent_runs	One row per agent invocation: status, timing, attempts, error context.
audit_events	Immutable, append-only event stream — never deleted.
patient_contexts	Cached OpenMRS chart snapshots fetched at encounter start.

Core HTTP API Endpoints

Full Swagger UI is exposed at <http://localhost:8000/docs>. The most important endpoints are listed below; ~40 endpoints total.

Method	Path	Purpose
POST	/encounters	Create a new encounter.
POST	/encounters/{id}/intake?auto_run=true	Upload audio and run the full auto-pipeline.
GET	/encounters/{id}/pipeline	Poll pipeline stage.
GET	/encounters/{id}/soap	Get current SOAP note.
PATCH	/encounters/{id}/soap	Update SOAP note (creates a new version).

POST	/encounters/{id}/review/open	Open physician review.
PATCH	/encounters/{id}/review/edit	Edit a single SOAP section.
POST	/encounters/{id}/review/approve	Approve (required before submit).
POST	/encounters/{id}/review/revert	Revert approval (only before submit).
POST	/encounters/{id}/submit	Submit approved encounter to OpenMRS via FHIR R4.
GET	/encounters/{id}/audit/timeline	Full chronological audit timeline.
GET	/agents	List all registered agents and their status.
GET	/agents/{name}/runs	Run history for a specific agent.
GET	/health	Health check.

5. Non-Functional Requirements

These requirements describe quality attributes of the system. They are referenced by ID elsewhere in the documentation.

ID	Requirement	Notes / Where Enforced
NFR-01	The full auto-pipeline (intake → transcription → SOAP → medications) shall complete within 3 minutes for a 10-minute audio recording under normal load.	Dependent on OpenAI API latency; not formally benchmarked under load (see Open Questions).
NFR-02	The system shall not allow any AI output to reach OpenMRS without physician approval.	Enforced server-side in PhysicianReviewAgent and the submission router; cannot be bypassed by the frontend.
NFR-03	The audit trail shall be append-only; no audit events shall be deletable via the API.	Enforced at the repository layer — there is no delete method on the audit repository.
NFR-04	All configuration shall be injectable via environment variables with no hard-coded secrets in source code.	config.py + .env pattern; .env is gitignored.
NFR-05	The backend API shall serve CORS-restricted responses; allowed origins shall be configurable.	Via the CORS_ORIGINS environment variable.
NFR-06	The system shall support retry with configurable backoff for transient agent failures.	AGENT_MAX_RETRIES, AGENT_RETRY_BASE_DELAY_SECONDS — handled by the orchestrator, not by individual agents.
NFR-07	Database schema changes shall be managed via Alembic migrations (no manual DDL).	9 migrations make up the current schema; alembic upgrade head applies them in order.
NFR-08	The system shall preserve all SOAP note versions and all physician edits for audit purposes.	Enforced by the soap_notes versioning model and the physician_edits table.
NFR-09	The system shall log which AI model and prompt version generated each SOAP note.	Stored on every soap_notes row; prompts are version-pinned in agents/prompts/.

6. General Schedule

ScribeGuard was developed across two sprints over the semester window. The plan below was a refinement of the original three-layer proposal in response to instructor feedback. Layer 1 (the ambient documentation MVP) was the deliverable scope; Layers 2 and 3 are documented as roadmap items but were not built.

Sprint 1 — Core Pipeline (Audio → SOAP Note)

Goals

By the end of Sprint 1, a physician should be able to upload an audio recording, see a Whisper transcript, and view a GPT-4 generated SOAP note in the UI.

Delivered

- FastAPI backend skeleton with PostgreSQL set up and connected via Docker Compose.

- Agent base class, AgentResult, AgentContext, and the central orchestrator implemented.
- EncounterIntakeAgent, TranscriptionAgent (Whisper), and ClinicalNoteGenerationAgent (GPT-4) implemented and integrated.
- React 19 + TypeScript dashboard with browser MediaRecorder audio capture wired to the intake endpoint.
- Initial PostgreSQL schema created with the first Alembic migration (encounters, transcripts, soap_notes, agent_runs, audit_events).
- GPT-4 system prompt engineered and tested for consistent, well-structured SOAP output across multiple test audio samples.
- Physician review screen with AI-assisted labeling and an explicit Approve & Submit button.

Sprint 2 — OpenMRS Integration, Entity Extraction & Polish

Goals

By the end of Sprint 2, the full audio-to-OpenMRS pipeline should be end-to-end demonstrable with a test patient in the OpenMRS sandbox, and the audit and review experience should be production-quality.

Delivered

- MedicationExtractionAgent and clinical entity extraction (allergies, conditions with ICD-10/SNOMED, vital signs, follow-ups).
- PhysicianReviewAgent with the open → edit → approve → revert state machine; per-section edit log persistence.
- OpenMRSIntegrationAgent split into auth, patient context, encounter mapper, note writer, and verifier sub-agents (so failures attribute to a specific stage in the audit trail).
- OpenMRS FHIR R4 write-back: Encounter, clinical-note Observation, AllergyIntolerance, Condition, MedicationRequest, vital-sign Observations — with returned UUIDs persisted on database rows.
- Encounter Workspace UI: SOAP editor, medication panel, allergy/condition/vitals/follow-up panels, agent timeline visualization.
- Live patient chart context sidebar fetching demographics, active medications, allergies, conditions, and recent encounters from OpenMRS at encounter start.
- Audit timeline page rendering the chronological event stream.
- Simulation mode (OPENMRS_SIMULATE=true) returning stable fake UUIDs for offline development and CI.
- Local AI provider option (faster-whisper + Ollama) for fully offline / free inference.
- Alembic migration suite completed (9 migrations) — schema is fully managed, no manual DDL required.
- End-to-end testing across at least three realistic clinical scenarios (diabetes follow-up, hypertension visit, post-surgical check-in).
- Bug fixes: OpenMRS submission step spinner, PDF attachment population, CORS configuration for development.

- Documentation package prepared for handoff: NEW_TEAM_HANDOFF.md, PROJECT_DEFINITION.md, PACKAGE_INDEX.md, ARCHITECTURE.md.

Scope Confirmation

- ☐ This sprint plan is fully within our original scope.
- ☒ This sprint plan represents a refinement and narrowing of our original scope.

Explanation of Refinement

The original proposal described a three-layer system comprising ambient note generation (Layer 1), prescription reconciliation (Layer 2), and dosage anomaly detection (Layer 3), along with both a React Native mobile application and a React web dashboard. Based on instructor feedback identifying the original scope as four to five semesters of work for a part-time team, the plan was scoped to Layer 1 only. The React Native mobile app was replaced with a web-based MediaRecorder implementation to eliminate an entire separate development effort. Layers 2 and 3 are documented as future roadmap items with architectural extension points stubbed in the codebase. The core clinical value proposition — eliminating manual SOAP note documentation and integrating with OpenMRS — is preserved entirely within this narrowed scope.

7. Healthcare Data, Standards, and Clinical Context

Healthcare Standards and Terminology

- **OpenMRS REST / FHIR2 R4 API** — patient demographics are read and clinical encounter data is written using the OpenMRS FHIR2 module. ScribeGuard talks to /openmrs/ws/fhir2/R4 and uses standard FHIR resource types: Encounter, Observation, AllergyIntolerance, Condition, MedicationRequest.
- **SOAP Note Format** — the universally accepted standard for clinical encounter documentation in U.S. outpatient and inpatient settings. The GPT-4 prompt is engineered to produce output conforming to this structure.
- **ICD-10 / SNOMED CT** — diagnosis codes captured with each Condition entity to support both billing and downstream FHIR mapping.
- **RxNorm (Future Layer)** — medication names extracted in the current MVP will be normalized to RxNorm concept codes in Layer 2's reconciliation engine, enabling standardized drug identification across provider systems.
- **OpenFDA Drug Labeling API (Future Layer)** — Layer 3's dosage safety validation is designed to use OpenFDA's publicly accessible drug labeling endpoint as the primary dosage threshold reference, intentionally avoiding commercial database dependencies.
- **Beers Criteria (Future Layer)** — reference for potentially inappropriate medications in older adults; planned for Layer 3.

Clinical Context

The project models a real, established clinical workflow: the post-encounter documentation session. Every physician in a U.S. outpatient clinic experiences this workflow multiple times per day. ScribeGuard does not invent a new workflow — it automates an existing one.

The OpenMRS sandbox instance used for development uses the OpenMRS Reference Application demo dataset, which contains synthetic patient records (encounter histories, medication orders, lab results) that provide realistic clinical context for integration testing without real patient data.

The GPT-4 prompt engineering draws on established clinical documentation guidelines from AHIMA (American Health Information Management Association) and CMS (Centers for Medicare & Medicaid Services) documentation requirements for evaluation and management coding, so generated notes reflect real clinical standards.

8. LOF Pillar Alignment

Primary Pillar

- ☒ Patient Engagement
- ☐ Data Analytics / Population Health
- ☐ Medical Education

Justification

ScribeGuard's primary user is the physician at the point of care — the moment of direct patient engagement. The system acts during and immediately after the patient encounter, capturing the conversation and transforming it into a clinical record that accurately reflects what was communicated, decided, and planned between the physician and the patient.

The healthcare problem ScribeGuard addresses — the loss of clinical information between the encounter and the written record — is a Patient Engagement problem at its core: when documentation is incomplete, inconsistent, or delayed, patients receive follow-up care based on an inaccurate record of what was actually discussed. ScribeGuard creates value within the Patient Engagement pillar by improving the fidelity and completeness of the clinical record at the source.

Evidence from the Implementation

Evidence 1 — Primary User Definition

Section 2d defines physicians and residents as the primary users with the explicit need for a documentation workflow that integrates naturally into the post-encounter routine without adding friction. The system is designed around the patient encounter as the unit of engagement.

Evidence 2 — OpenMRS Write-Back

Section 2e Step 6 describes how the finalized SOAP note is written back into the patient's OpenMRS record immediately after physician approval. The patient's engagement record — what was discussed, diagnosed, and planned in their visit — is updated in real time, improving the accuracy of data that every future care interaction will be based on. This is a direct contribution to patient-centered health data management.

Evidence 3 — AI-Assisted with Physician Control

Sections 2c and 2d explicitly require that no note is ever auto-committed without physician approval, and that every note is clearly labeled as AI-assisted and editable before submission. This design ensures the physician remains engaged and accountable in the documentation process — consistent with the Patient Engagement pillar's emphasis on meaningful, human-centered care interactions rather than automated replacement of clinician judgment.

9. Submission of Results / Deliverables

The complete handoff package contains everything required for a new team to pick up the project and continue development.

Deliverables Index

Deliverable	Location	Purpose
Source Code	Repository root: backend/, frontend/	Complete, runnable application — Layer 1 MVP.
Database Migrations	backend/alembic/	9 Alembic migrations — full schema, applied via alembic upgrade head.
Sample Data / Audio	test_transcripts/	Sample audio files for pipeline testing.
AI Prompts	backend/app/agents/prompts/	Version-pinned GPT-4 prompts. Do not edit casually — version constants must be incremented.
Docker Configuration	docker-compose.yml	PostgreSQL + (optional) backend container.
Setup & Build Guide	Documentation/NEW_TEAM_HANDOFF.md	Step-by-step environment setup, run, build, known issues, gotchas.
Project Definition	Documentation/PROJECT_DEFINITION.md (and this Word document)	Filled-out CS595 template covering problem, scope, requirements, sprint history.
Package Index	Documentation/PACKAGE_INDEX.md	TL;DR overview and read order for the documentation package.
Engineering Reference	ARCHITECTURE.md	Deep-dive: agent contracts, state machine, extension points, design rationale.
Quick-Start Overview	README.md	5-minute project overview at the repository root.
Original Proposal	Documentation/annotated-ProjectProposal.pdf	Original three-layer proposal (preserved for context).
Revised Proposal	Documentation/annotated-ProjectProposalAddendum.pdf	Scope-narrowed proposal aligned with the delivered Layer 1 MVP.
Original Template	Documentation/CS595 2026 — Project Info Template.docx	Blank template originally provided by the instructor.

How a New Team Picks Up the Project

The recommended read order is captured in PACKAGE_INDEX.md and reproduced here:

- Read NEW_TEAM_HANDOFF.md first — it is the operational guide. Sections 3–5 cover prerequisites, environment setup, and running the application; Sections 6–11 cover code structure, agent pipeline, database, and the API surface; Sections 12–14 cover extending the system.
- Read this Project Definition document second for the full “why” behind the project: problem framing, scope decisions, requirement IDs, and sprint history.
- Read README.md third for the quickest possible orientation.

- Keep ARCHITECTURE.md open as a reference while writing code — it documents agent contracts, the state machine, the repository pattern, and the explicit extension points for Layers 2 and 3.

Fastest-Possible Setup

Reproduced from PACKAGE_INDEX.md. Full detail in NEW_TEAM_HANDOFF.md Section 4.

```
git clone <repo-url> && cd ScribeGuard docker compose up -d db cd backend && python -m venv venv
&& source venv/bin/activate pip install -r requirements.txt # Create backend/.env (template in
NEW_TEAM_HANDOFF.md §4) alembic upgrade head uvicorn app.main:app --reload --port 8000 #
Terminal 1 cd ../frontend && npm install && npm run dev # Terminal 2
```

Open <http://localhost:5173> (UI) and <http://localhost:8000/docs> (Swagger) to verify everything is running.

10. Open Questions / Items Reserved for the New Team

Per the template guidance — “If you have any blanks, that's perfectly fine” — the items below are intentional gaps. They were either out of MVP scope, out of available time, or require physician/clinical input that was not arranged during the semester. Each is a starting point for the new team.

Production Deployment Architecture

Not yet designed. The system runs locally; a production hosting plan (cloud provider, container orchestration, TLS termination, secrets management, CI/CD) has not been specified.

HIPAA Compliance Audit

The audit trail design was inspired by HIPAA requirements (immutable, append-only, with full edit history), but a formal compliance review was not conducted. A real deployment would require encryption at rest, key management, and a documented Business Associate Agreement chain with OpenAI (or a switch to local-only inference via Ollama).

Layer 2 — Prescription Reconciliation Engine

Architectural extension points are stubbed. Concrete starting point: `backend/app/openmrs/medication.py` already has helpers for reading existing `MedicationRequest` resources from OpenMRS. A new `ReconciliationAgent` should run after `MedicationExtractionAgent`, compare extracted medications against OpenMRS chart medications, and flag duplicates, therapeutic conflicts, and outdated prescriptions. A new `reconciliation_findings` table is required.

Layer 3 — Dosage Anomaly Detector

Depends on Layer 2's reconciled medication list. External APIs to integrate: OpenFDA Drug Labeling (free public) and optionally DrugBank (license required). The `DosageAnomalyAgent` should validate each medication's dose against patient-specific clinical parameters (age, weight, eGFR, recent labs) read from OpenMRS.

Performance Benchmarking

NFR-01 sets a 3-minute pipeline target for a 10-minute audio recording, but this was not formally measured under load. The new team should benchmark against multiple audio lengths and OpenAI rate-limit conditions.

User Acceptance Testing

No formal usability testing with real physicians was conducted. The current UI is based on design assumptions about physician workflows. A round of think-aloud sessions with attending physicians or residents would validate or refute these assumptions.

Authentication / Authorization

The current system has a stub login page but no real authentication. Production would need OAuth2 / OpenID Connect, ideally federated with OpenMRS user accounts so a physician's identity in ScribeGuard matches the Practitioner UUID used for FHIR writes.

Data Retention Policy

How long audio files, transcripts, draft SOAP versions, and audit events should be retained was not defined. Audio in particular is privacy-sensitive and should likely be auto-purged after a configurable retention window once submission to OpenMRS is verified.

Mobile Application

Was descoped early. The HTTP API is mobile-ready; a React Native app could be added later without backend changes. This is currently the lowest-priority item.

End of Project Definition Document

ScribeGuard — CS595-LOF-2026-001 — Final Submission

See NEW_TEAM_HANDOFF.md, PACKAGE_INDEX.md, and ARCHITECTURE.md for the operational handoff package.